

# 01 — Arrays & Strings

DSA in Python — Interview Prep

---

## Arrays & Strings — Tutorial

Covers the 10 problems solved in this folder. Arrays/strings aren't a single technique — this topic is really a grab-bag of five reusable patterns that show up constantly in later topics too.

### The patterns

#### 1. Mirror two-pointer (01, 09)

One pointer at index 0, one at index `len-1`, walking inward until they meet or cross. Used to process a sequence from both ends at once — reversal, palindrome checks.

```
left, right = 0, len(s) - 1
while left < right:
    # look at / swap s[left], s[right]
    left += 1
    right -= 1
```

`09_valid_palindrome` extends this with inner skip-loops (ignore non-alphanumeric chars) and case-insensitive comparison — same skeleton, extra filtering per step.

#### 2. Read/write pointer — in-place partition/compaction (02, 03)

One pointer (`i/read`) scans every element; a second pointer (`insert_pos/write`) only advances when the current element needs to be kept. This is how you filter or dedupe an array without allocating a new one.

- `move_zeroes`: write pointer advances on non-zero elements, swap.
- `remove_duplicates_sorted_array`: write pointer advances when the current element differs from the \*last written\* element (relies on the array being sorted so duplicates are adjacent).

#### 3. Single-pass running state (04, 07)

Track one or two running values while scanning once, left to right. No lookahead, no extra storage proportional to input size.

- `max_subarray` (Kadane's): running sum, reset to the current element whenever the running sum goes negative (dead weight).
- `best_time_to_buy_sell_stock`: running minimum price + running best profit, checked in that order each step.

This is the simplest form of dynamic programming: "the answer at position `i` depends only on the answer at position `i-1`," computed without an explicit DP array since only the last state is needed.

#### 4. Prefix / suffix precomputation (06)

When you need "everything before `i`" and "everything after `i`" combined, and can't do it in one pass, do it in two: build a prefix result left-to-right, then fold in a suffix result right-to-left over the same output array.

`product_except_self` is the canonical example — also the standard answer to "compute X without using operation Y" (here: product without division).

### 5. Two-pointer merge from the correct end (05, 08)

- `rotate_array`: the "three reversals" trick — reverse the whole array, then reverse each of the two resulting segments — turns an array rotation into three  $O(n)$  passes with zero extra space.
- `merge_sorted_array`: when one array has trailing free space, merge **from the back** (largest elements first) instead of the front, avoiding the  $O(n)$  per-insert shifting a forward merge would need.

### 6. Vertical / structural scan (10)

`longest_common_prefix` scans by \*character position\* across all strings simultaneously, rather than comparing strings pairwise. Generalizes to any "find the common structure across N sequences" problem.

## Complexity summary

| #  | Problem                                     | Time           | Extra space           |
|----|---------------------------------------------|----------------|-----------------------|
| 01 | <code>reverse_string</code>                 | $O(n)$         | $O(1)$                |
| 02 | <code>move_zeroes</code>                    | $O(n)$         | $O(1)$                |
| 03 | <code>remove_duplicates_sorted_array</code> | $O(n)$         | $O(1)$                |
| 04 | <code>max_subarray</code>                   | $O(n)$         | $O(1)$                |
| 05 | <code>rotate_array</code>                   | $O(n)$         | $O(1)$                |
| 06 | <code>product_except_self</code>            | $O(n)$         | $O(1)$ (excl. output) |
| 07 | <code>best_time_to_buy_sell_stock</code>    | $O(n)$         | $O(1)$                |
| 08 | <code>merge_sorted_array</code>             | $O(m+n)$       | $O(1)$                |
| 09 | <code>valid_palindrome</code>               | $O(n)$         | $O(1)$                |
| 10 | <code>longest_common_prefix</code>          | $O(n \cdot m)$ | $O(1)$ (excl. output) |

Every problem in this set is linear time, constant extra space — that combination (in-place, single or double pass) is the recurring interview expectation for "easy" array/string problems. If an approach needs a new array/dict proportional to input size, that's usually a sign there's a more in-place way to do it (as in 02, 03, 05, 08 above), \*unless\* the problem genuinely needs a hash map — which is exactly what topic 02\_hashing covers next.

## Common interview follow-ups to be ready for

- "Can you do it in  $O(1)$  space?" — already true for all 10 here; know \*why\* for each (which pointer trick makes it possible).
- `max_subarray`: "return the actual subarray, not just the sum" — track start/end indices alongside the running sum.
- `rotate_array`: "what if k is negative (rotate left)?" — normalize with  $k \% n$  first, then a negative k still works with Python's modulo, but be ready to explain rotating left vs right.
- `merge_sorted_array`: "what if nums1 doesn't have extra space?" — becomes a different problem (merge into new array, or in-place merge using  $O(1)$  space with a harder swap-based algorithm).

- `product_except_self`: "what if the array is streamed / can't fit in memory?" — no longer solvable in  $O(1)$  space; discuss  $O(n)$  as the floor once random access is gone.
- `longest_common_prefix`: "what about a trie instead?" — vertical scan is  $O(n \cdot m)$  worst case; a trie built once amortizes better if you'll query prefixes repeatedly (preview of topic `09_tries`).

## Self-check before moving on

You should be able to, without looking at the code:

- Write the mirror two-pointer skeleton from memory.
- Explain why `remove_duplicates_sorted_array`'s approach breaks if the array isn't sorted.
- Explain why Kadane's algorithm resets instead of just tracking the global max naively.
- Derive the three-reversal rotation trick on paper with a 5-element example.

If any of those aren't fluent yet, this topic isn't "done" per the README's definition — revisit before moving to `02_hashing`.

# Programs

## 01. Reverse String

01\_reverse\_string.py

### *Explanation*

Problem: Reverse a list of characters in place.

Source : LeetCode 344 - Reverse String

Given a list of characters `s`, reverse it in place using  $O(1)$  extra space (don't return a new list, don't use `s[::-1]` or `list(reversed(s))` - the point is to practice the two-pointer technique by hand).

Example:

Input: ['h','e','l','l','o']

Output: ['o','l','l','e','h']

### *Code*

```
def reverse_string(s: list[str]) -> None:
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

if __name__ == "__main__":
    tests = [
        (["h", "e", "l", "l", "o"], ["o", "l", "l", "e", "h"]),
        (["H", "a", "n", "n", "a", "h"], ["h", "a", "n", "n", "a", "H"]),
        ([], []),
        (["a"], ["a"]),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        arr = inp[:] # copy, since we mutate in place
        reverse_string(arr)
        status = "PASS" if arr == expected else "FAIL"
        print(f"Test {i}: {status} (got {arr}, expected {expected})")
```

## 02. Move Zeroes

02\_move\_zeroes.py

### *Explanation*

Problem: Move all zeroes in an array to the end, in place, while keeping the relative order of the non-zero elements. Must be done in  $O(1)$  extra space (no new list, no sorting).

Source : LeetCode 283 - Move Zeroes

Example:

Input: [0, 1, 0, 3, 12]

Output: [1, 3, 12, 0, 0]

### Code

```
def move_zeroes(nums: list[int]) -> None:
    insert_pos = 0
    for i in range(len(nums)):
        if nums[i] != 0:
            nums[insert_pos], nums[i] = nums[i], nums[insert_pos]
            insert_pos += 1

if __name__ == "__main__":
    tests = [
        ([0, 1, 0, 3, 12], [1, 3, 12, 0, 0]),
        ([0, 0, 1], [1, 0, 0]),
        ([1, 2, 3], [1, 2, 3]),
        ([0, 0, 0], [0, 0, 0]),
        ([], []),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        arr = inp[:]
        move_zeroes(arr)
        status = "PASS" if arr == expected else "FAIL"
        print(f"Test {i}: {status} (got {arr}, expected {expected})")
```

## 03. Remove Duplicates Sorted Array

03\_remove\_duplicates\_sorted\_array.py

### Explanation

Problem: Given a sorted array **nums**, remove duplicates in place so each unique element appears once. Return k, the number of unique elements; the first k slots of nums must hold those elements (order preserved). O(1) extra space, no new list.

Source : LeetCode 26 - Remove Duplicates from Sorted Array

Example:

Input: [0, 0, 1, 1, 1, 2, 2, 3, 3, 4]

Output: k = 5, nums[:5] == [0, 1, 2, 3, 4]

### Code

```
def remove_duplicates(nums: list[int]) -> int:
    if not nums:
        return 0

    write = 1
    for read in range(1, len(nums)):
        if nums[read] != nums[write - 1]:
            nums[write] = nums[read]
            write += 1
    return write

if __name__ == "__main__":
    tests = [
```

```

([1, 1, 2], 2, [1, 2]),
([0, 0, 1, 1, 1, 2, 2, 3, 3, 4], 5, [0, 1, 2, 3, 4]),
([], 0, []),
([1], 1, [1]),
([1, 2, 3], 3, [1, 2, 3]),
]

for i, (inp, expected_k, expected_prefix) in enumerate(tests, 1):
    arr = inp[:]
    k = remove_duplicates(arr)
    got_prefix = arr[:k]
    status = "PASS" if (k == expected_k and got_prefix == expected_prefix) else "FAIL"
    print(f"Test {i}: {status} (k={k}, prefix={got_prefix}, expected k={expected_k}, prefix={expected_

```

## 04. Max Subarray

04\_max\_subarray.py

### Explanation

Problem: Given an integer array **nums**, find the contiguous subarray (containing at least one number) with the largest sum, and return that sum. Classic intro to Kadane's algorithm / 1D DP-as-single-pass.  
Source : LeetCode 53 - Maximum Subarray

Example:

Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

Output: 6 (subarray [4, -1, 2, 1])

### Code

```

def max_subarray(nums: list[int]) -> int:
    max_sum = nums[0]
    curr_sum = nums[0]

    for n in nums[1:]:
        # either extend the running subarray, or start fresh at n -
        # whichever gives a bigger sum ending exactly at this index
        curr_sum = max(n, curr_sum + n)
        max_sum = max(max_sum, curr_sum)

    return max_sum

if __name__ == "__main__":
    tests = [
        ([-2, 1, -3, 4, -1, 2, 1, -5, 4], 6),
        ([1], 1),
        ([5, 4, -1, 7, 8], 23),
        ([-1], -1),
        ([-2, -1], -1),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = max_subarray(inp[:])
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")

```

## 05. Rotate Array

05\_rotate\_array.py

### Explanation

Problem: Rotate an array to the right by k steps, in place, using O(1) extra space. Solved with the "three reversals" trick.

Source : LeetCode 189 - Rotate Array

Example:

Input: nums = [1, 2, 3, 4, 5, 6, 7], k = 3

Output: [5, 6, 7, 1, 2, 3, 4]

Idea: reversing the whole array puts the last k elements at the front (but each half is internally backwards), then reversing each half individually fixes the internal order.

[1,2,3,4,5,6,7]

-> reverse all: [7,6,5,4,3,2,1]

-> reverse first k: [5,6,7,4,3,2,1]

-> reverse remaining: [5,6,7,1,2,3,4]

### Code

```
def _reverse(nums: list[int], left: int, right: int) -> None:
    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1

def rotate_array(nums: list[int], k: int) -> None:
    n = len(nums)
    if n == 0:
        return

    k %= n
    _reverse(nums, 0, n - 1)
    _reverse(nums, 0, k - 1)
    _reverse(nums, k, n - 1)

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 4, 5, 6, 7], 3, [5, 6, 7, 1, 2, 3, 4]),
        ([-1, -100, 3, 99], 2, [3, 99, -1, -100]),
        ([1, 2], 3, [2, 1]),
        ([1], 0, [1]),
        ([], 5, []),
    ]

    for i, (inp, k, expected) in enumerate(tests, 1):
        arr = inp[:]
        rotate_array(arr, k)
        status = "PASS" if arr == expected else "FAIL"
        print(f"Test {i}: {status} (got {arr}, expected {expected})")
```

## 06. Product Except Self

06\_product\_except\_self.py

### Explanation

Problem: Given an integer array **nums**, return an array **output** where **output[i]** is the product of every element except **nums[i]**.

Must run in  $O(n)$  time, without using division.

Source : LeetCode 238 - Product of Array Except Self

Example:

Input: [1, 2, 3, 4]

Output: [24, 12, 8, 6]

Idea: **output[i]** = (product of everything to the left of i)

\* (product of everything to the right of i)

Build the left-products in one left-to-right pass, then fold in the right-products in one right-to-left pass, reusing the same output array (so only  $O(1)$  \*extra\* space beyond the output itself).

### Code

```
def product_except_self(nums: list[int]) -> list[int]:
    n = len(nums)
    result = [1] * n

    prefix = 1
    for i in range(n):
        result[i] = prefix
        prefix *= nums[i]

    suffix = 1
    for i in range(n - 1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]

    return result

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 4], [24, 12, 8, 6]),
        ([-1, 1, 0, -3, 3], [0, 0, 9, 0, 0]),
        ([2, 3], [3, 2]),
        ([5], [1]),
        ([0, 0], [0, 0]),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = product_except_self(inp[:])
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 07. Best Time To Buy Sell Stock

07\_best\_time\_to\_buy\_sell\_stock.py

### Explanation



Problem: Given an array `prices` where `prices[i]` is the stock price on day `i`, find the maximum profit from a single buy followed by a single sell (buy must happen before sell). Return 0 if no profit is possible. Single pass,  $O(1)$  extra space.

Source : LeetCode 121 - Best Time to Buy and Sell Stock

Example:

Input: [7, 1, 5, 3, 6, 4]

Output: 5 (buy at 1, sell at 6)

### Code

```
def max_profit(prices: list[int]) -> int:
    if not prices:
        return 0

    min_price = prices[0]
    best_profit = 0

    for p in prices[1:]:
        best_profit = max(best_profit, p - min_price)
        min_price = min(min_price, p)

    return best_profit

if __name__ == "__main__":
    tests = [
        ([7, 1, 5, 3, 6, 4], 5),
        ([7, 6, 4, 3, 1], 0),
        ([2, 4, 1], 2),
        ([], 0),
        ([5], 0),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = max_profit(inp[:])
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 08. Merge Sorted Array

08\_merge\_sorted\_array.py

### Explanation

Problem: `nums1` and `nums2` are sorted arrays. `nums1` has length  $m + n$ : the first  $m$  elements are the real values, the last  $n$  are placeholder zeros with room for `nums2`'s  $n$  elements. Merge `nums2` into `nums1` in place so `nums1` becomes one sorted array of length  $m + n$ .

Source : LeetCode 88 - Merge Sorted Array

Example:

`nums1 = [1, 2, 3, 0, 0, 0]`,  $m = 3$

`nums2 = [2, 5, 6]`,  $n = 3$

-> `nums1` becomes `[1, 2, 2, 3, 5, 6]`

Idea: merging from the front needs shifting ( $O(n)$  per insert). Since `nums1` has empty room at the *\*back\**, merge from the back instead: compare the largest remaining elements of each array and place the bigger one at the last open slot, working backwards.

### Code

```
def merge_sorted_array(nums1: list[int], m: int, nums2: list[int], n: int) -> None:
    i = m - 1          # last real element in nums1
    j = n - 1          # last element in nums2
    k = m + n - 1      # last write position in nums1

    while j >= 0:
        if i >= 0 and nums1[i] > nums2[j]:
            nums1[k] = nums1[i]
            i -= 1
        else:
            nums1[k] = nums2[j]
            j -= 1
        k -= 1

if __name__ == "__main__":
    tests = [
        ([1, 2, 3, 0, 0, 0], 3, [2, 5, 6], 3, [1, 2, 2, 3, 5, 6]),
        ([1], 1, [], 0, [1]),
        ([0], 0, [1], 1, [1]),
        ([2, 0], 1, [1], 1, [1, 2]),
        ([4, 5, 6, 0, 0, 0], 3, [1, 2, 3], 3, [1, 2, 3, 4, 5, 6]),
    ]

    for i, (nums1, m, nums2, n, expected) in enumerate(tests, 1):
        arr = nums1[:]
        merge_sorted_array(arr, m, nums2, n)
        status = "PASS" if arr == expected else "FAIL"
        print(f"Test {i}: {status} (got {arr}, expected {expected})")
```

## 09. Valid Palindrome

09\_valid\_palindrome.py

### Explanation

Problem: Given a string `s`, determine if it is a palindrome after considering only alphanumeric characters and ignoring case.

Source : LeetCode 125 - Valid Palindrome

Example:

Input: "A man, a plan, a canal: Panama"

Output: True (reads "amanaplanacanalpanama" both ways)

Input: "race a car"

Output: False

Idea: same mirror two-pointer shape as `reverse_string`, but each pointer independently skips non-alphanumeric characters before the comparison, and comparison is case-insensitive.

## Code

```
def is_palindrome(s: str) -> bool:
    left, right = 0, len(s) - 1

    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1

        if s[left].lower() != s[right].lower():
            return False

        left += 1
        right -= 1

    return True

if __name__ == "__main__":
    tests = [
        ("A man, a plan, a canal: Panama", True),
        ("race a car", False),
        ("", True),
        (" ", True),
        (".", True),
        ("0P", False),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = is_palindrome(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got}, expected {expected})")
```

## 10. Longest Common Prefix

10\_longest\_common\_prefix.py

### Explanation

Problem: Given a list of strings, find the longest common prefix shared by all of them. Return "" if there is no common prefix.

Source : LeetCode 14 - Longest Common Prefix

Example:

Input: ["flower", "flow", "flight"]

Output: "fl"

Idea: vertical scan. Walk character-index by character-index using the first string as the reference; at each index, check that every other string has the same character there. Stop at the first mismatch (or the first string that's too short) and the prefix so far is the answer.

## Code

```
def longest_common_prefix(strs: list[str]) -> str:
    if not strs:
        return ""
```

```

    for i, char in enumerate(strs[0]):
        for s in strs[1:]:
            if i >= len(s) or s[i] != char:
                return strs[0][:i]

    return strs[0]

if __name__ == "__main__":
    tests = [
        (["flower", "flow", "flight"], "fl"),
        (["dog", "racecar", "car"], ""),
        (["a"], "a"),
        ([], ""),
        (["", "b"], ""),
        (["interspecies", "interstellar", "interstate"], "inters"),
    ]

    for i, (inp, expected) in enumerate(tests, 1):
        got = longest_common_prefix(inp)
        status = "PASS" if got == expected else "FAIL"
        print(f"Test {i}: {status} (got {got!r}, expected {expected!r})")

```